

My Project

Sugeneruota Doxygen 1.15.0

1 Hierarchijos Indeksas	1
1.1 Klasių hierarchija	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Klasės Dokumentacija	7
4.1 Studentas Struktūra	7
4.1.1 Atributų Dokumentacija	7
4.1.1.1 egz	7
4.1.1.2 gal	7
4.1.1.3 med	7
4.1.1.4 pav	7
4.1.1.5 paz	8
4.1.1.6 var	8
4.2 Studentas_klase Klasė	8
4.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	9
4.2.1.1 Studentas_klase() [1/3]	9
4.2.1.2 Studentas_klase() [2/3]	9
4.2.1.3 ~Studentas_klase()	9
4.2.1.4 Studentas_klase() [3/3]	9
4.2.2 Metodų Dokumentacija	9
4.2.2.1 egzaminas()	9
4.2.2.2 galutinis()	9
4.2.2.3 mediana_galutinis()	10
4.2.2.4 operator=()	10
4.2.2.5 pavarde()	10
4.2.2.6 pazymiai()	10
4.2.2.7 skaiciuok_galutinius()	10
4.2.2.8 skaityk_studenta_class()	10
4.2.2.9 vardas()	10
4.2.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	10
4.2.3.1 operator<<	10
4.2.3.2 operator>>	11
4.3 Zmogus Klasė	11
4.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	11
4.3.1.1 Zmogus() [1/2]	11
4.3.1.2 ~Zmogus()	11
4.3.1.3 Zmogus() [2/2]	12
4.3.2 Metodų Dokumentacija	12

4.3.2.1 operator=()	12
4.3.2.2 pavarde()	12
4.3.2.3 skaityk_studenta_class()	12
4.3.2.4 vardas()	12
4.3.3 Atributų Dokumentacija	12
4.3.3.1 pav_	12
4.3.3.2 var_	12
5 Failo Dokumentacija	13
5.1 Generuoti_failai.cpp Failo Nuoroda	13
5.1.1 Funkcijos Dokumentacija	13
5.1.1.1 failu_generavimas()	13
5.1.1.2 partition_palyginimas()	13
5.2 Generuoti_failai.h Failo Nuoroda	14
5.2.1 Funkcijos Dokumentacija	14
5.2.1.1 ar_naudoti_shrink_to_fit() [1/2]	14
5.2.1.2 ar_naudoti_shrink_to_fit() [2/2]	15
5.2.1.3 failu_generavimas()	15
5.2.1.4 pagalbine_funkc_2_strategija() [1/2]	15
5.2.1.5 pagalbine_funkc_2_strategija() [2/2]	15
5.2.1.6 pagalbine_funkcija() [1/2]	15
5.2.1.7 pagalbine_funkcija() [2/2]	15
5.2.1.8 partition_palyginimas()	15
5.2.1.9 skaitymas_is_generuoto_failo()	16
5.2.1.10 studentu_rusiavimas()	16
5.3 Generuoti_failai.h	16
5.4 Generuoti_failai_class.cpp Failo Nuoroda	22
5.5 Generuoti_failai_class.h Failo Nuoroda	23
5.5.1 Funkcijos Dokumentacija	23
5.5.1.1 ar_naudoti_shrink_to_fit_class() [1/2]	23
5.5.1.2 ar_naudoti_shrink_to_fit_class() [2/2]	24
5.5.1.3 pagalbine_funkc_2_strategija_class() [1/2]	24
5.5.1.4 pagalbine_funkc_2_strategija_class() [2/2]	24
5.5.1.5 pagalbine_funkcija_class() [1/2]	24
5.5.1.6 pagalbine_funkcija_class() [2/2]	24
5.5.1.7 skaitymas_is_generuoto_failo_class()	24
5.5.1.8 studentu_rusiavimas_class()	24
5.6 Generuoti_failai_class.h	25
5.7 is_number.cpp Failo Nuoroda	31
5.7.1 Funkcijos Dokumentacija	31
5.7.1.1 isNumber()	31
5.8 is_number.h Failo Nuoroda	31

5.8.1 Funkcijos Dokumentacija	31
5.8.1.1 isNumber()	31
5.9 is_number.h	32
5.10 mediana.cpp Failo Nuoroda	32
5.10.1 Funkcijos Dokumentacija	32
5.10.1.1 Rask_mediana()	32
5.11 mediana.h Failo Nuoroda	32
5.11.1 Funkcijos Dokumentacija	32
5.11.1.1 Rask_mediana()	32
5.12 mediana.h	33
5.13 myMultiFileApp.cpp Failo Nuoroda	33
5.13.1 Funkcijos Dokumentacija	33
5.13.1.1 main()	33
5.14 palyginimas.cpp Failo Nuoroda	33
5.14.1 Funkcijos Dokumentacija	34
5.14.1.1 palyginimas_2_strategija()	34
5.14.1.2 palyginimas_galutinis()	34
5.14.1.3 palyginimas_pavarde()	34
5.14.1.4 palyginimas_vardas()	34
5.15 palyginimas.h Failo Nuoroda	34
5.15.1 Funkcijos Dokumentacija	34
5.15.1.1 palyginimas_2_strategija()	34
5.15.1.2 palyginimas_galutinis()	35
5.15.1.3 palyginimas_pavarde()	35
5.15.1.4 palyginimas_vardas()	35
5.16 palyginimas.h	35
5.17 palyginimas_class.cpp Failo Nuoroda	35
5.17.1 Funkcijos Dokumentacija	35
5.17.1.1 palyginimas_2_strategija_class()	35
5.17.1.2 palyginimas_galutinis_class()	36
5.17.1.3 palyginimas_pavarde_class()	36
5.17.1.4 palyginimas_vardas_class()	36
5.17.1.5 partition_palyginimas_class()	36
5.18 palyginimas_class.h Failo Nuoroda	36
5.18.1 Funkcijos Dokumentacija	36
5.18.1.1 palyginimas_2_strategija_class()	36
5.18.1.2 palyginimas_galutinis_class()	37
5.18.1.3 palyginimas_pavarde_class()	37
5.18.1.4 palyginimas_vardas_class()	37
5.18.1.5 partition_palyginimas_class()	37
5.19 palyginimas_class.h	37
5.20 Simple.cpp Failo Nuoroda	37

5.20.1 Funkcijos Dokumentacija	38
5.20.1.1 Stud_iv()	38
5.20.1.2 Stud_iv_atstitiktinai()	38
5.21 Simple.h Failo Nuoroda	38
5.21.1 Funkcijos Dokumentacija	38
5.21.1.1 rasydas()	38
5.21.1.2 skaitymas()	38
5.21.1.3 Stud_iv()	39
5.21.1.4 Stud_iv_atstitiktinai()	39
5.22 Simple.h	39
5.23 Simple_class.cpp Failo Nuoroda	41
5.23.1 Funkcijos Dokumentacija	41
5.23.1.1 Stud_iv_atstitiktinai_class()	41
5.23.1.2 Stud_iv_class()	41
5.24 Simple_class.h Failo Nuoroda	41
5.24.1 Funkcijos Dokumentacija	42
5.24.1.1 rasydas_class()	42
5.24.1.2 skaitymas_class()	42
5.24.1.3 Stud_iv_atstitiktinai_class()	42
5.24.1.4 Stud_iv_class()	42
5.25 Simple_class.h	42
5.26 Studentas.h Failo Nuoroda	43
5.27 Studentas.h	44
5.28 Studentas_klase.cpp Failo Nuoroda	44
5.28.1 Funkcijos Dokumentacija	44
5.28.1.1 operator<<()	44
5.28.1.2 operator>>()	44
5.29 Studentas_klase.h Failo Nuoroda	45
5.30 Studentas_klase.h	45
Rodyklė	47

skyrius 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Studentas	7
Zmogus	11
Studentas_klase	8

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	7
Studentas_klase	8
Zmogus	11

skyrius 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

Generuoti_failai.cpp	13
Generuoti_failai.h	14
Generuoti_failai_class.cpp	22
Generuoti_failai_class.h	23
is_number.cpp	31
is_number.h	31
mediana.cpp	32
mediana.h	32
myMultiFileApp.cpp	33
palyginimas.cpp	33
palyginimas.h	34
palyginimas_class.cpp	35
palyginimas_class.h	36
Simple.cpp	37
Simple.h	38
Simple_class.cpp	41
Simple_class.h	41
Studentas.h	43
Studentas_klase.cpp	44
Studentas_klase.h	45

skyrius 4

Klasės Dokumentacija

4.1 Studentas Struktūra

```
#include <Studentas.h>
```

Vieši Atributai

- string `var`
- string `pav`
- vector< int > `paz`
- int `egz`
- double `gal`
- double `med`

4.1.1 Atributų Dokumentacija

4.1.1.1 egz

```
int Studentas::egz
```

4.1.1.2 gal

```
double Studentas::gal
```

4.1.1.3 med

```
double Studentas::med
```

4.1.1.4 pav

```
string Studentas::pav
```

4.1.1.5 paz

```
vector<int> Studentas::paz
```

4.1.1.6 var

```
string Studentas::var
```

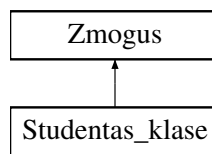
Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [Studentas.h](#)

4.2 Studentas_klase Klasė

```
#include <Studentas_klase.h>
```

Paveldimumo diagrama Studentas_klase:



Vieši Metodai

- [Studentas_klase](#) ()
- [Studentas_klase](#) (istream &is)
- string [vardas](#) () const
- string [pavarde](#) () const
- double [galutinis](#) () const
- double [mediana_galutinis](#) () const
- const vector< int > & [pazymiai](#) () const
- int [egzaminas](#) () const
- istream & [skaityk_studenta_class](#) (istream &is)
- void [skaiciuok_galutinius](#) ()
- [~Studentas_klase](#) ()
- [Studentas_klase](#) (const [Studentas_klase](#) &s)
- [Studentas_klase](#) & [operator=](#) (const [Studentas_klase](#) &s)

Vieši Metodai inherited from [Zmogus](#)

- [Zmogus](#) ()
- virtual [~Zmogus](#) ()
- [Zmogus](#) (const [Zmogus](#) &z)
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &z)

Draugai

- `std::istream & operator>>` (`std::istream &in`, `Studentas_klase &s`)
- `std::ostream & operator<<` (`std::ostream &out`, `const Studentas_klase &s`)

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- `string var_`
- `string pav_`

4.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.2.1.1 Studentas_klase() [1/3]

```
Studentas_klase::Studentas_klase ()
```

4.2.1.2 Studentas_klase() [2/3]

```
Studentas_klase::Studentas_klase (  
    istream & is)
```

4.2.1.3 ~Studentas_klase()

```
Studentas_klase::~~Studentas_klase ()
```

4.2.1.4 Studentas_klase() [3/3]

```
Studentas_klase::Studentas_klase (  
    const Studentas_klase & s)
```

4.2.2 Metodu Dokumentacija

4.2.2.1 egzaminas()

```
int Studentas_klase::egzaminas () const [inline]
```

4.2.2.2 galutinis()

```
double Studentas_klase::galutinis () const [inline]
```

4.2.2.3 mediana_galutinis()

```
double Studentas_klase::mediana_galutinis () const [inline]
```

4.2.2.4 operator=()

```
Studentas_klase & Studentas_klase::operator= (  
    const Studentas_klase & s)
```

4.2.2.5 pavarde()

```
string Studentas_klase::pavarde () const [inline], [virtual]
```

Realizuoja [Zmogus](#).

4.2.2.6 pazymiai()

```
const vector< int > & Studentas_klase::pazymiai () const [inline]
```

4.2.2.7 skaiciuok_galutinius()

```
void Studentas_klase::skaiciuok_galutinius ()
```

4.2.2.8 skaityk_studenta_class()

```
istream & Studentas_klase::skaityk_studenta_class (  
    istream & is) [virtual]
```

Realizuoja [Zmogus](#).

4.2.2.9 vardas()

```
string Studentas_klase::vardas () const [inline], [virtual]
```

Realizuoja [Zmogus](#).

4.2.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.2.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Studentas_klase & s) [friend]
```


4.2.3.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas_klase & s) [friend]
```

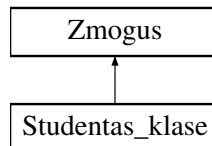
Dokumentacija šiai klasei sugeneruota iš šių failų:

- [Studentas_klase.h](#)
- [Studentas_klase.cpp](#)

4.3 Zmogus Klasė

```
#include <Studentas_klase.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus \(\)](#)
- virtual string [vardas \(\)](#) const =0
- virtual string [pavarde \(\)](#) const =0
- virtual istream & [skaityk_studenta_class](#) (istream &is)=0
- virtual [~Zmogus \(\)](#)
- [Zmogus](#) (const [Zmogus](#) &z)
- [Zmogus & operator=](#) (const [Zmogus](#) &z)

Apsaugoti Atributai

- string [var_](#)
- string [pav_](#)

4.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.3.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus ()
```

4.3.1.2 ~Zmogus()

```
Zmogus::~Zmogus () [virtual]
```

4.3.1.3 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const Zmogus & z)
```

4.3.2 Metodų Dokumentacija

4.3.2.1 operator=()

```
Zmogus & Zmogus::operator= (
    const Zmogus & z)
```

4.3.2.2 pavarde()

```
virtual string Zmogus::pavarde () const [inline], [pure virtual]
```

Realizuota [Studentas_klase](#).

4.3.2.3 skaityk_studenta_class()

```
virtual istream & Zmogus::skaityk_studenta_class (
    istream & is) [pure virtual]
```

Realizuota [Studentas_klase](#).

4.3.2.4 vardas()

```
virtual string Zmogus::vardas () const [inline], [pure virtual]
```

Realizuota [Studentas_klase](#).

4.3.3 Atributų Dokumentacija

4.3.3.1 pav_

```
string Zmogus::pav_ [protected]
```

4.3.3.2 var_

```
string Zmogus::var_ [protected]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [Studentas_klase.h](#)
- [Studentas_klase.cpp](#)

skyrius 5

Failo Dokumentacija

5.1 Generuoti_failai.cpp Failo Nuoroda

```
#include "Generuoti_failai.h"
```

Funkcijos

- void `failu_generavimas` (int `k`, int `pazymiu_sk`)
- bool `partition_palyginimas` (const `Studentas` &`s`)

5.1.1 Funkcijos Dokumentacija

5.1.1.1 failu_generavimas()

```
void failu_generavimas (  
    int k,  
    int pazymiu_sk)
```

5.1.1.2 partition_palyginimas()

```
bool partition_palyginimas (  
    const Studentas & s)
```

5.2 Generuoti_failai.h Failo Nuoroda

```
#include <iostream>
#include "Studentas.h"
#include <fstream>
#include <sstream>
#include <iomanip>
#include <numeric>
#include "mediana.h"
#include "is_number.h"
#include <cstdlib>
#include <chrono>
#include "palyginimas.h"
#include <random>
#include <vector>
#include <list>
#include <algorithm>
#include <string>
```

Funkcijos

- void [failu_generavimas](#) (int k, int pazymiu_sk)
- template<typename T>
void [pagalbine_funkcija](#) (vector< T > &Grupe, int pasirinkimas)
- template<typename T>
void [pagalbine_funkcija](#) (list< T > &Grupe, int pasirinkimas)
- template<typename T>
void [pagalbine_funkc_2_strategija](#) (vector< T > &Grupe)
- template<typename T>
void [pagalbine_funkc_2_strategija](#) (list< T > &Grupe)
- template<typename T>
void [ar_naudoti_shrink_to_fit](#) (vector< T > &Grupe)
- template<typename T>
void [ar_naudoti_shrink_to_fit](#) (list< T > &Grupe)
- bool [partition_palyginimas](#) (const [Studentas](#) &s)
- template<typename T>
void [studentu_rusiavimas](#) (T &Grupe, int k, double &diff_rusiavimas1, double &diff_irasu_dalijimo1, double &diff_irasyms_i_vargsiuku_faila1, double &diff_irasyms_i_kietiaku_faila1, int i, int rusiavimo_strategija, const string &spr)
- template<typename T>
void [skaitymas_is_generuoto_failo](#) (T &Grupe, const string &failo_vardas, double &diff_skaitymas1)

5.2.1 Funkcijos Dokumentacija

5.2.1.1 [ar_naudoti_shrink_to_fit\(\)](#) [1/2]

```
template<typename T>
void ar_naudoti_shrink_to_fit (
    list< T > & Grupe)
```

5.2.1.2 ar_naudoti_shrink_to_fit() [2/2]

```
template<typename T>
void ar_naudoti_shrink_to_fit (
    vector< T > & Grupe)
```

5.2.1.3 failu_generavimas()

```
void failu_generavimas (
    int k,
    int pazymiu_sk)
```

5.2.1.4 pagalbine_funkc_2_strategija() [1/2]

```
template<typename T>
void pagalbine_funkc_2_strategija (
    list< T > & Grupe)
```

5.2.1.5 pagalbine_funkc_2_strategija() [2/2]

```
template<typename T>
void pagalbine_funkc_2_strategija (
    vector< T > & Grupe)
```

5.2.1.6 pagalbine_funkcija() [1/2]

```
template<typename T>
void pagalbine_funkcija (
    list< T > & Grupe,
    int pasirinkimas)
```

5.2.1.7 pagalbine_funkcija() [2/2]

```
template<typename T>
void pagalbine_funkcija (
    vector< T > & Grupe,
    int pasirinkimas)
```

5.2.1.8 partition_palyginimas()

```
bool partition_palyginimas (
    const Studentas & s)
```

5.2.1.9 skaitymas_is_generuoto_failo()

```
template<typename T>
void skaitymas_is_generuoto_failo (
    T & Grupe,
    const string & failo_vardas,
    double & diff_skaitymas1)
```

5.2.1.10 studentu_rusiavimas()

```
template<typename T>
void studentu_rusiavimas (
    T & Grupe,
    int k,
    double & diff_rusiavimas1,
    double & diff_irasu_dalijimol,
    double & diff_irasymas_i_vargsiuku_failal,
    double & diff_irasymas_i_kietiakus_failal,
    int i,
    int rusiavimo_strategija,
    const string & spr)
```

5.3 Generuoti_failai.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include <iostream>
00003 #include "Studentas.h"
00004 #include <fstream>
00005 #include <sstream>
00006 #include <iomanip>
00007 #include <numeric>
00008 #include "mediana.h"
00009 #include "is_number.h"
00010 #include <cstdlib>
00011 #include <chrono>
00012 #include "palyginimas.h"
00013 #include <random>
00014
00015 #include <vector>
00016 #include <list>
00017 #include <algorithm> // sort funkcijai
00018 #include <string>
00019
00020 using std::cout;
00021 using std::cin;
00022 using std::endl;
00023 using std::setw;
00024 using std::stringstream;
00025 using std::left;
00026 using std::right;
00027 using std::ofstream;
00028 using std::ifstream;
00029 using std::accumulate;
00030 using std::to_string;
00031 using std::fixed;
00032 using std::setprecision;
00033 using std::chrono::high_resolution_clock;
00034 using std::chrono::duration;
00035
00036 using std::stable_partition;
00037 using std::copy;
00038 using std::distance;
00039 using std::reverse;
00040
00041 void failu_generavimas(int k, int pazymiu_sk);
00042
```

```

00043 template <typename T> void pagalbine_funkcija(vector <T>& Grupe, int pasirinkimas) {
00044     if (pasirinkimas == 1) {
00045         sort(Grupe.begin(), Grupe.end(), palyginimas_vardas);
00046     }
00047     else if (pasirinkimas == 2) {
00048         sort(Grupe.begin(), Grupe.end(), palyginimas_pavarde);
00049     }
00050     else if (pasirinkimas == 3) {
00051         sort(Grupe.begin(), Grupe.end(), palyginimas_galutinis);
00052     }
00053 }
00054
00055 template <typename T> void pagalbine_funkcija(list <T>& Grupe, int pasirinkimas) {
00056     if (pasirinkimas == 1) {
00057         Grupe.sort(palyginimas_vardas);
00058     }
00059     else if (pasirinkimas == 2) {
00060         Grupe.sort(palyginimas_pavarde);
00061     }
00062     else if (pasirinkimas == 3) {
00063         Grupe.sort(palyginimas_galutinis);
00064     }
00065 }
00066
00067 template <typename T> void pagalbine_funkc_2_strategija(vector <T>& Grupe) {
00068     sort(Grupe.begin(), Grupe.end(), palyginimas_2_strategija);
00069 }
00070
00071 template <typename T> void pagalbine_funkc_2_strategija(list <T>& Grupe) {
00072     Grupe.sort(palyginimas_2_strategija);
00073 }
00074
00075 template <typename T> void ar_naudoti_shrink_to_fit(vector <T>& Grupe) {
00076     Grupe.shrink_to_fit();
00077 }
00078
00079 template <typename T> void ar_naudoti_shrink_to_fit(list <T>& Grupe) {
00080 }
00081
00082 bool partition_palyginimas(const Studentas& s);
00083
00084 template <typename T> void studentu_rusiavimas(T& Grupe, int k,
00085     double& diff_rusiavimas1, double& diff_irasu_dalijimo1,
00086     double& diff_irasymas_i_vargsiuku_failai,
00087     double& diff_irasymas_i_kietiakui_failai, int i, int rusiavimo_strategija, const string& spr) {
00088
00089     int pasirinkimas = i;
00090
00091     auto start_rusiavimas = high_resolution_clock::now();
00092     pagalbine_funkcija(Grupe, pasirinkimas);
00093     duration<double> diff_rusiavimas = high_resolution_clock::now() - start_rusiavimas;
00094
00095     cout << k << " irasu rusiavimo didejimo tvarka laikas, su sort funkcija: " << diff_rusiavimas.count()
00096     << " s" << endl;
00097     diff_rusiavimas1 = diff_rusiavimas.count();
00098     T vargsiukai;
00099     T kietiakai;
00100
00101     //Irasu dalijimas 1 strategija:
00102     if (rusiavimo_strategija == 1) {
00103         vargsiukai.clear();
00104         kietiakai.clear();
00105         auto start_irasu_dalijimo = high_resolution_clock::now();
00106         for (const auto& studentas : Grupe) {
00107             if (studentas.gal < 5.0) {
00108                 vargsiukai.push_back(studentas);
00109             }
00110             else {
00111                 kietiakai.push_back(studentas);
00112             }
00113         }
00114         duration<double> diff_irasu_dalijimo = high_resolution_clock::now() - start_irasu_dalijimo;
00115         cout << k << " irasu dalijimo i dvi grupes laikas 1 strategija: " << diff_irasu_dalijimo.count()
00116         << " s" << endl;
00117         diff_irasu_dalijimo1 = diff_irasu_dalijimo.count();
00118
00119         auto start_irasymas_i_vargsiuku_faila = high_resolution_clock::now();
00120         stringstream vargsiukai_buffer, kietiakai_buffer;
00121         if (spr == "vidurki") {
00122             vargsiukai_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
00123             left << "Galutinis (Vid.)" << endl;
00124             for (int i = 0; i < 58; i++) {
00125                 vargsiukai_buffer << "-";
00126             }
00127             vargsiukai_buffer << endl;
00128             for (const auto& studentas : vargsiukai) {
00129                 vargsiukai_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav

```

```

    « setw(18) « left « fixed « setprecision(2) « studentas.gal « endl;
00127 }
00128 }
00129 else if (spr == "mediana") {
00130     vargsiukai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « endl;
00131     for (int i = 0; i < 58; i++) {
00132         vargsiukai_buffer « "-";
00133     }
00134     vargsiukai_buffer « endl;
00135     for (const auto& studentas : vargsiukai) {
00136         vargsiukai_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav
« setw(18) « left « fixed « setprecision(2) « studentas.med « endl;
00137     }
00138 }
00139 else if (spr == "abu") {
00140     vargsiukai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « setw(18) « left « "Galutinis (Med.)" « endl;
00141     for (int i = 0; i < 76; i++) {
00142         vargsiukai_buffer « "-";
00143     }
00144     vargsiukai_buffer « endl;
00145     for (const auto& studentas : vargsiukai) {
00146         vargsiukai_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav
« setw(18) « left « fixed « setprecision(2) « studentas.gal « setw(18) « left « fixed «
setprecision(2) « studentas.med « endl;
00147     }
00148 }
00149 string failo_vardas1 = "vargsiukai" + to_string(k) + ".txt";
00150 ofstream vargsiukai_failas(failo_vardas1);
00151 vargsiukai_failas « vargsiukai_buffer.str();
00152 vargsiukai_failas.close();
00153 duration<double> diff_irasymas_i_vargsiuku_faila = high_resolution_clock::now() -
start_irasymas_i_vargsiuku_faila;
00154 cout « k « " irasu irasymo i vargsiuku faila laikas: " «
diff_irasymas_i_vargsiuku_faila.count() « " s" « endl;
00155 diff_irasymas_i_vargsiuku_faila1 = diff_irasymas_i_vargsiuku_faila.count();
00156
00157 auto start_irasymas_i_kietiakui_faila = high_resolution_clock::now();
00158 /*kietiakui_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Vid.)" « endl;
00159     for (int i = 0; i < 58; i++) {
00160         kietiakui_buffer « "-";
00161     }
00162     kietiakui_buffer « endl;
00163     for (const auto& studentas : kietiakui) {
00164         kietiakui_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
setw(18) « left « fixed « setprecision(2) « studentas.gal « endl;
00165     }*/
00166     if (spr == "vidurki") {
00167         kietiakui_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Vid.)" « endl;
00168         for (int i = 0; i < 58; i++) {
00169             kietiakui_buffer « "-";
00170         }
00171         kietiakui_buffer « endl;
00172         for (const auto& studentas : kietiakui) {
00173             kietiakui_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
setw(18) « left « fixed « setprecision(2) « studentas.gal « endl;
00174         }
00175     }
00176     else if (spr == "mediana") {
00177         kietiakui_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « endl;
00178         for (int i = 0; i < 58; i++) {
00179             kietiakui_buffer « "-";
00180         }
00181         kietiakui_buffer « endl;
00182         for (const auto& studentas : kietiakui) {
00183             kietiakui_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
setw(18) « left « fixed « setprecision(2) « studentas.med « endl;
00184         }
00185     }
00186     else if (spr == "abu") {
00187         kietiakui_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « setw(18) « left « "Galutinis (Med.)" « endl;
00188         for (int i = 0; i < 76; i++) {
00189             kietiakui_buffer « "-";
00190         }
00191         kietiakui_buffer « endl;
00192         for (const auto& studentas : kietiakui) {
00193             kietiakui_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
« studentas.med « endl;
00194         }
00195     }
00196     string failo_vardas2 = "kietiakui" + to_string(k) + ".txt";

```



```

00197         ofstream kietiakai_failas(failo_vardas2);
00198         kietiakai_failas << kietiakai_buffer.str();
00199         kietiakai_failas.close();
00200         duration<double> diff_irasyimas_i_kietiaiku_faila = high_resolution_clock::now() -
start_irasyimas_i_kietiaiku_faila;
00201         cout << k << " irasu irasyimo i kietiaiku faila laikas: " << diff_irasyimas_i_kietiaiku_faila.count()
<< " s" << endl;
00202         diff_irasyimas_i_kietiaiku_faila1 = diff_irasyimas_i_kietiaiku_faila.count();
00203
00204         cout << "Surusiuota studentu:" << endl;
00205         cout << "Vargsai (< 5.0): " << vargsiukai.size() << " studentai" << endl;
00206         cout << "Kietiakai (>= 5.0): " << kietiakai.size() << " studentai" << endl;
00207     }
00208     //Irasu dalijimo 1 strategijos pabaiga
00209
00210     //Irasu dalijimas 2 strategija:
00211     else if (rusiavimo_strategija == 2) {
00212         vargsiukai.clear();
00213         kietiakai.clear();
00214         auto start_irasu_dalijimo = high_resolution_clock::now();
00215         pagalbine_funkc_2_strategija(Grupe);
00216         while (!Grupe.empty() && Grupe.back().gal < 5.0) {
00217             vargsiukai.push_back(Grupe.back());
00218             Grupe.pop_back();
00219         }
00220         ar_naudoti_shrink_to_fit(Grupe);
00221         pagalbine_funkcija(Grupe, pasirinkimas);
00222         pagalbine_funkcija(vargsiukai, pasirinkimas);
00223         duration<double> diff_irasu_dalijimo = high_resolution_clock::now() - start_irasu_dalijimo;
00224         cout << k << " irasu dalijimo i dvi grupes laikas 2 strategija: " << diff_irasu_dalijimo.count()
<< " s" << endl;
00225         diff_irasu_dalijimo1 = diff_irasu_dalijimo.count();
00226
00227
00228
00229         auto start_irasyimas_i_vargsiuku_faila = high_resolution_clock::now();
00230         stringstream vargsiukai_buffer, kietiakai_buffer;
00231         if (spr == "vidurki") {
00232             vargsiukai_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Vid.)" << endl;
00233             for (int i = 0; i < 58; i++) {
00234                 vargsiukai_buffer << "-";
00235             }
00236             vargsiukai_buffer << endl;
00237             for (const auto& studentas : vargsiukai) {
00238                 vargsiukai_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav
<< setw(18) << left << fixed << setprecision(2) << studentas.gal << endl;
00239             }
00240         }
00241         else if (spr == "mediana") {
00242             vargsiukai_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Med.)" << endl;
00243             for (int i = 0; i < 58; i++) {
00244                 vargsiukai_buffer << "-";
00245             }
00246             vargsiukai_buffer << endl;
00247             for (const auto& studentas : vargsiukai) {
00248                 vargsiukai_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav
<< setw(18) << left << fixed << setprecision(2) << studentas.med << endl;
00249             }
00250         }
00251         else if (spr == "abu") {
00252             vargsiukai_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Med.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00253             for (int i = 0; i < 76; i++) {
00254                 vargsiukai_buffer << "-";
00255             }
00256             vargsiukai_buffer << endl;
00257             for (const auto& studentas : vargsiukai) {
00258                 vargsiukai_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav
<< setw(18) << left << fixed << setprecision(2) << studentas.gal << setw(18) << left << fixed <<
setprecision(2) << studentas.med << endl;
00259             }
00260         }
00261
00262         string failo_vardas1 = "vargsiukai" + to_string(k) + ".txt";
00263         ofstream vargsiukai_failas(failo_vardas1);
00264         vargsiukai_failas << vargsiukai_buffer.str();
00265         vargsiukai_failas.close();
00266         duration<double> diff_irasyimas_i_vargsiuku_faila = high_resolution_clock::now() -
start_irasyimas_i_vargsiuku_faila;
00267         cout << k << " irasu irasyimo i vargsiuku faila laikas: " <<
diff_irasyimas_i_vargsiuku_faila.count() << " s" << endl;
00268         diff_irasyimas_i_vargsiuku_faila1 = diff_irasyimas_i_vargsiuku_faila.count();
00269
00270         auto start_irasyimas_i_kietiaiku_faila = high_resolution_clock::now();
00271         if (spr == "vidurki") {

```

```

00272         kietiakai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Vid.)" « endl;
00273         for (int i = 0; i < 58; i++) {
00274             kietiakai_buffer « "-";
00275         }
00276         kietiakai_buffer « endl;
00277         for (const auto& studentas : Grupe) {
00278             kietiakai_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
setw(18) « left « fixed « setprecision(2) « studentas.gal « endl;
00279         }
00280     }
00281     else if (spr == "mediana") {
00282         kietiakai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « endl;
00283         for (int i = 0; i < 58; i++) {
00284             kietiakai_buffer « "-";
00285         }
00286         kietiakai_buffer « endl;
00287         for (const auto& studentas : Grupe) {
00288             kietiakai_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
setw(18) « left « fixed « setprecision(2) « studentas.med « endl;
00289         }
00290     }
00291     else if (spr == "abu") {
00292         kietiakai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « setw(18) « left « "Galutinis (Med.)" « endl;
00293         for (int i = 0; i < 76; i++) {
00294             kietiakai_buffer « "-";
00295         }
00296         kietiakai_buffer « endl;
00297         for (const auto& studentas : Grupe) {
00298             kietiakai_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav «
setw(18) « left « fixed « setprecision(2) « studentas.gal « setw(18) « left « fixed « setprecision(2)
« studentas.med « endl;
00299         }
00300     }
00301
00302     string failo_vardas2 = "kietiakai" + to_string(k) + ".txt";
00303     ofstream kietiakai_failas(failo_vardas2);
00304     kietiakai_failas « kietiakai_buffer.str();
00305     kietiakai_failas.close();
00306     duration<double> diff_irasymas_i_kietiakui_faila = high_resolution_clock::now() -
start_irasymas_i_kietiakui_faila;
00307     cout « k « " irasu irasymo i kietiakui faila laikas: " « diff_irasymas_i_kietiakui_faila.count()
« " s" « endl;
00308     diff_irasymas_i_kietiakui_faila1 = diff_irasymas_i_kietiakui_faila.count();
00309
00310     cout « "Surusiuta studentu:" « endl;
00311     cout « "Vargsai (< 5.0): " « vargsiukai.size() « " studentai" « endl;
00312     cout « "Kietiakai (>= 5.0): " « Grupe.size() « " studentai" « endl;
00313 }
00314 //Irasu dalijimo 2 strategijos pabaiga
00315
00316 //Irasu dalijimas 3 strategija:
00317 else if (rusiavimo_strategija == 3) {
00318     vargsiukai.clear();
00319     kietiakai.clear();
00320     auto start_irasu_dalijimo = high_resolution_clock::now();
00321
00322     auto vidurys = stable_partition(Grupe.begin(), Grupe.end(), partition_palyginimas);
00323     vargsiukai.resize(distance(vidurys, Grupe.end()));
00324     copy(vidurys, Grupe.end(), vargsiukai.begin());
00325     Grupe.erase(vidurys, Grupe.end());
00326     duration<double> diff_irasu_dalijimo = high_resolution_clock::now() - start_irasu_dalijimo;
00327     cout « k « " irasu dalijimo i dvi grupes laikas 3 strategija: " « diff_irasu_dalijimo.count()
« " s" « endl;
00328     diff_irasu_dalijimo1 = diff_irasu_dalijimo.count();
00329
00330     ar_naudoti_shrink_to_fit(Grupe);
00331
00332     auto start_irasymas_i_vargsiuku_faila = high_resolution_clock::now();
00333     stringstream vargsiukai_buffer, kietiakai_buffer;
00334     if (spr == "vidurki") {
00335         vargsiukai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Vid.)" « endl;
00336         for (int i = 0; i < 58; i++) {
00337             vargsiukai_buffer « "-";
00338         }
00339         vargsiukai_buffer « endl;
00340         for (const auto& studentas : vargsiukai) {
00341             vargsiukai_buffer « setw(20) « left « studentas.var « setw(20) « left « studentas.pav
« setw(18) « left « fixed « setprecision(2) « studentas.gal « endl;
00342         }
00343     }
00344     else if (spr == "mediana") {
00345         vargsiukai_buffer « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « endl;

```

```

00346         for (int i = 0; i < 58; i++) {
00347             vargsiukai_buffer << "-";
00348         }
00349         vargsiukai_buffer << endl;
00350         for (const auto& studentas : vargsiukai) {
00351             vargsiukai_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav
<< setw(18) << left << fixed << setprecision(2) << studentas.med << endl;
00352         }
00353     }
00354     else if (spr == "abu") {
00355         vargsiukai_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Med.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00356         for (int i = 0; i < 76; i++) {
00357             vargsiukai_buffer << "-";
00358         }
00359         vargsiukai_buffer << endl;
00360         for (const auto& studentas : vargsiukai) {
00361             vargsiukai_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav
<< setw(18) << left << fixed << setprecision(2) << studentas.gal << setw(18) << left << fixed <<
setprecision(2) << studentas.med << endl;
00362         }
00363     }
00364
00365     string failo_vardas1 = "vargsiukai" + to_string(k) + ".txt";
00366     ofstream vargsiukai_failas(failo_vardas1);
00367     vargsiukai_failas << vargsiukai_buffer.str();
00368     vargsiukai_failas.close();
00369     duration<double> diff_irasymas_i_vargsiuku_faila = high_resolution_clock::now() -
start_irasymas_i_vargsiuku_faila;
00370     cout << k << " irasu irasyimo i vargsiuku faila laikas: " <<
diff_irasymas_i_vargsiuku_faila.count() << " s" << endl;
00371     diff_irasymas_i_vargsiuku_faila1 = diff_irasymas_i_vargsiuku_faila.count();
00372
00373     auto start_irasymas_i_kietiakui_faila = high_resolution_clock::now();
00374     if (spr == "vidurki") {
00375         kietiakui_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Vid.)" << endl;
00376         for (int i = 0; i < 58; i++) {
00377             kietiakui_buffer << "-";
00378         }
00379         kietiakui_buffer << endl;
00380         for (const auto& studentas : Grupe) {
00381             kietiakui_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav <<
setw(18) << left << fixed << setprecision(2) << studentas.gal << endl;
00382         }
00383     }
00384     else if (spr == "mediana") {
00385         kietiakui_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Med.)" << endl;
00386         for (int i = 0; i < 58; i++) {
00387             kietiakui_buffer << "-";
00388         }
00389         kietiakui_buffer << endl;
00390         for (const auto& studentas : Grupe) {
00391             kietiakui_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav <<
setw(18) << left << fixed << setprecision(2) << studentas.med << endl;
00392         }
00393     }
00394     else if (spr == "abu") {
00395         kietiakui_buffer << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Med.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00396         for (int i = 0; i < 76; i++) {
00397             kietiakui_buffer << "-";
00398         }
00399         kietiakui_buffer << endl;
00400         for (const auto& studentas : Grupe) {
00401             kietiakui_buffer << setw(20) << left << studentas.var << setw(20) << left << studentas.pav <<
setw(18) << left << fixed << setprecision(2) << studentas.gal << setw(18) << left << fixed << setprecision(2)
<< studentas.med << endl;
00402         }
00403     }
00404
00405     string failo_vardas2 = "kietiakui" + to_string(k) + ".txt";
00406     ofstream kietiakui_failas(failo_vardas2);
00407     kietiakui_failas << kietiakui_buffer.str();
00408     kietiakui_failas.close();
00409     duration<double> diff_irasymas_i_kietiakui_faila = high_resolution_clock::now() -
start_irasymas_i_kietiakui_faila;
00410     cout << k << " irasu irasyimo i kietiakui faila laikas: " << diff_irasymas_i_kietiakui_faila.count()
<< " s" << endl;
00411     diff_irasymas_i_kietiakui_faila1 = diff_irasymas_i_kietiakui_faila.count();
00412
00413     cout << "Surusiuota studentu:" << endl;
00414     cout << "Vargsai (< 5.0): " << vargsiukai.size() << " studentai" << endl;
00415     cout << "Kietiakai (>= 5.0): " << Grupe.size() << " studentai" << endl;
00416 }
00417 //Irasu dalijimo 3 strategijos pabaiga

```

```

00418 }
00419
00420 template <typename T> void skaitymas_is_generuoto_failo(T& Grupe, const string& failo_vardas, double&
diff_skaitymas1) {
00421     auto start_skaitymas = high_resolution_clock::now();
00422     ifstream F(failo_vardas);
00423     if (!F) {
00424         cout << "Klaida: Nepavyko atidaryti failo " << failo_vardas << endl;
00425         exit(0);
00426     }
00427     string pavadinimai;
00428     getline(F, pavadinimai);
00429     Studentas Pirmas;
00430     int skait = 0;
00431     while (F.peek() != EOF) {
00432         F >> Pirmas.var >> Pirmas.pav;
00433         if (F.fail() || Pirmas.var.size() == 0 || Pirmas.pav.size() == 0) {
00434             F.clear();
00435             break;
00436         }
00437         skait++;
00438         float mediana;
00439         int sum = 0;
00440         Pirmas.paz.clear();
00441         while (F.peek() != '\n' && F.peek() != EOF) {
00442             int pazymys;
00443             string pazymys_pr;
00444             F >> pazymys_pr;
00445             if (isNumber(pazymys_pr)) {
00446                 if (stoi(pazymys_pr) >= 1 && stoi(pazymys_pr) <= 10) {
00447                     pazymys = stoi(pazymys_pr);
00448                     Pirmas.paz.push_back(pazymys);
00449                 }
00450                 else {
00451                     cout << "Studento nr. " << skait << " pazymiuse buvo klaida (ne sveikasis skaicius
nuo 1 iki 10): " << pazymys_pr << ". Klaida pasalinta is skaiciavimo" << endl;
00452                 }
00453             }
00454             else {
00455                 cout << "Studento nr. " << skait << " pazymiuse buvo klaida (ne sveikasis skaicius nuo 1
iki 10): " << pazymys_pr << ". Klaida pasalinta is skaiciavimo" << endl;
00456             }
00457         }
00458         if (Pirmas.paz.size() != 0) {
00459             Pirmas.egz = Pirmas.paz.back();
00460             Pirmas.paz.pop_back();
00461         }
00462         else {
00463             Pirmas.egz = 0;
00464         }
00465         sum = accumulate(Pirmas.paz.begin(), Pirmas.paz.end(), 0);
00466         int n;
00467         n = Pirmas.paz.size();
00468         if (Pirmas.paz.size() == 0) {
00469             Pirmas.gal = Pirmas.egz * 0.6;
00470             Pirmas.med = Pirmas.egz * 0.6;
00471         }
00472         else {
00473             Pirmas.gal = double(sum) / double(n) * 0.4 + Pirmas.egz * 0.6;
00474             mediana = Rask_mediana(Pirmas.paz);
00475             Pirmas.med = mediana * 0.4 + Pirmas.egz * 0.6;
00476         }
00477         Grupe.push_back(Pirmas);
00478         Pirmas.paz.clear();
00479     }
00480     F.close();
00481     duration<double> diff_skaitymas = high_resolution_clock::now() - start_skaitymas;
00482     cout << "\nFailo is " << Grupe.size() << " irasu nuskaitymo laikas: " << diff_skaitymas.count() << " s"
<< endl;
00483     diff_skaitymas1 = diff_skaitymas.count();
00484     cout << "Duomenys nuskaityti is failo. Rastas studentu skaicius: " << Grupe.size() << endl;
00485 }

```

5.4 Generuoti_failai_class.cpp Failo Nuoroda

```

#include "Generuoti_failai_class.h"
#include "Studentas_klase.h"
#include "palyginimas_class.h"
#include <fstream>

```

```
#include <sstream>
#include <iomanip>
#include <chrono>
#include <algorithm>
#include <numeric>
#include <iostream>
```

5.5 Generuoti_failai_class.h Failo Nuoroda

```
#include <vector>
#include <string>
#include <list>
#include "Studentas_klase.h"
#include <fstream>
#include <sstream>
#include <iomanip>
#include <chrono>
#include <algorithm>
#include <numeric>
#include <iostream>
#include "is_number.h"
#include "palyginimas_class.h"
```

Funkcijos

- `template<typename T>`
void [pagalbine_funkcija_class](#) (vector< T > &Grupe, int pasirinkimas)
- `template<typename T>`
void [pagalbine_funkcija_class](#) (list< T > &Grupe, int pasirinkimas)
- `template<typename T>`
void [pagalbine_funkc_2_strategija_class](#) (vector< T > &Grupe)
- `template<typename T>`
void [pagalbine_funkc_2_strategija_class](#) (list< T > &Grupe)
- `template<typename T>`
void [ar naudoti shrink_to_fit_class](#) (vector< T > &Grupe)
- `template<typename T>`
void [ar naudoti shrink_to_fit_class](#) (list< T > &Grupe)
- `template<typename T>`
void [studentu_rusiavimas_class](#) (T &Grupe, int k, double &diff_rusiavimas1, double &diff_irasu_dalijimo1, double &diff_irasymas_i_vargsiuku_faila1, double &diff_irasymas_i_kietiaiku_faila1, int i, int rusiavimo_↵ strategija, const string &spr)
- `template<typename T>`
void [skaitymas_is_generuoto_failo_class](#) (T &Grupe, const string &failo_vardas, double &diff_skaitymas1)

5.5.1 Funkcijos Dokumentacija

5.5.1.1 ar naudoti shrink_to_fit_class() [1/2]

```
template<typename T>
void ar naudoti shrink_to_fit_class (
    list< T > & Grupe)
```

5.5.1.2 ar_naudoti_shrink_to_fit_class() [2/2]

```
template<typename T>
void ar_naudoti_shrink_to_fit_class (
    vector< T > & Grupe)
```

5.5.1.3 pagalbine_funkc_2_strategija_class() [1/2]

```
template<typename T>
void pagalbine_funkc_2_strategija_class (
    list< T > & Grupe)
```

5.5.1.4 pagalbine_funkc_2_strategija_class() [2/2]

```
template<typename T>
void pagalbine_funkc_2_strategija_class (
    vector< T > & Grupe)
```

5.5.1.5 pagalbine_funkcija_class() [1/2]

```
template<typename T>
void pagalbine_funkcija_class (
    list< T > & Grupe,
    int pasirinkimas)
```

5.5.1.6 pagalbine_funkcija_class() [2/2]

```
template<typename T>
void pagalbine_funkcija_class (
    vector< T > & Grupe,
    int pasirinkimas)
```

5.5.1.7 skaitymas_is_generuoto_failo_class()

```
template<typename T>
void skaitymas_is_generuoto_failo_class (
    T & Grupe,
    const string & failo_vardas,
    double & diff_skaitymas1)
```

5.5.1.8 studentu_rusiavimas_class()

```
template<typename T>
void studentu_rusiavimas_class (
    T & Grupe,
    int k,
    double & diff_rusiavimas1,
    double & diff_irasu_dalijimol,
    double & diff_irasymas_i_vargsiuku_failal,
    double & diff_irasymas_i_kietiakus_failal,
    int i,
    int rusiavimo_strategija,
    const string & spr)
```

5.6 Generuoti_failai_class.h

Eiti į šio failo dokumentaciją.

```

00001 #pragma once
00002 #include <vector>
00003 #include <string>
00004 #include <list>
00005 #include "Studentas_klase.h"
00006 #include <fstream>
00007 #include <sstream>
00008 #include <iomanip>
00009 #include <chrono>
00010 #include <algorithm>
00011 #include <numeric>
00012 #include <iostream>
00013 #include "is_number.h"
00014
00015 #include "palyginimas_class.h"
00016
00017 using std::vector;
00018 using std::string;
00019 using std::ifstream;
00020 using std::ofstream;
00021 using std::stringstream;
00022 using std::cout;
00023 using std::endl;
00024 using std::setw;
00025 using std::left;
00026 using std::right;
00027 using std::fixed;
00028 using std::setprecision;
00029 using std::to_string;
00030 using std::chrono::high_resolution_clock;
00031 using std::chrono::duration;
00032 using std::sort;
00033 using std::getline;
00034
00035 template <typename T> void pagalbine_funkcija_class(vector <T>& Grupe, int pasirinkimas) {
00036     if (pasirinkimas == 1) {
00037         sort(Grupe.begin(), Grupe.end(), palyginimas_vardas_class);
00038     }
00039     else if (pasirinkimas == 2) {
00040         sort(Grupe.begin(), Grupe.end(), palyginimas_pavarde_class);
00041     }
00042     else if (pasirinkimas == 3) {
00043         sort(Grupe.begin(), Grupe.end(), palyginimas_galutinis_class);
00044     }
00045 }
00046
00047 template <typename T> void pagalbine_funkcija_class(list <T>& Grupe, int pasirinkimas) {
00048     if (pasirinkimas == 1) {
00049         Grupe.sort(palyginimas_vardas_class);
00050     }
00051     else if (pasirinkimas == 2) {
00052         Grupe.sort(palyginimas_pavarde_class);
00053     }
00054     else if (pasirinkimas == 3) {
00055         Grupe.sort(palyginimas_galutinis_class);
00056     }
00057 }
00058
00059 template <typename T> void pagalbine_funkc_2_strategija_class(vector <T>& Grupe) {
00060     sort(Grupe.begin(), Grupe.end(), palyginimas_2_strategija_class);
00061 }
00062
00063 template <typename T> void pagalbine_funkc_2_strategija_class(list <T>& Grupe) {
00064     Grupe.sort(palyginimas_2_strategija_class);
00065 }
00066
00067 template <typename T> void ar_naudoti_shrink_to_fit_class(vector <T>& Grupe) {
00068     Grupe.shrink_to_fit();
00069 }
00070
00071 template <typename T> void ar_naudoti_shrink_to_fit_class(list <T>& Grupe) {
00072 }
00073
00074 // Studentu rusiavimas tik vektoriais
00075 template <typename T> void studentu_rusiavimas_class(T& Grupe,
00076     int k,
00077     double& diff_rusiavimas1,
00078     double& diff_irasu_dalijimol,
00079     double& diff_irasymas_i_vargsiuku_failal,
00080     double& diff_irasymas_i_kietiaiku_failal,
00081     int i,
00082     int rusiavimo_strategija, const string& spr) {

```

```

00083
00084     int pasirinkimas = i;
00085     auto start_rusiavimas = high_resolution_clock::now();
00086
00087     pagalbine_funkcija_class(Grupe, pasirinkimas);
00088
00089     duration<double> diff_rusiavimas = high_resolution_clock::now() - start_rusiavimas;
00090     cout << k << " irasu rusiavimo didejimo tvarka laikas, su sort funkcija: " << diff_rusiavimas.count()
00091     << " s" << endl;
00092     diff_rusiavimas1 = diff_rusiavimas.count();
00093
00094     T vargsiukai;
00095     T kietiakai;
00096
00097     // 1 strategija
00098     if (rusiavimo_strategija == 1) {
00099         vargsiukai.clear();
00100         kietiakai.clear();
00101         auto start_irasu_dalijimo = high_resolution_clock::now();
00102         for (const auto& studentas : Grupe) {
00103             if (studentas.galutinis() < 5.0) {
00104                 vargsiukai.push_back(studentas);
00105             }
00106             else {
00107                 kietiakai.push_back(studentas);
00108             }
00109             duration<double> diff_irasu_dalijimo = high_resolution_clock::now() - start_irasu_dalijimo;
00110             cout << k << " irasu dalijimo i dvi grupes laikas 1 strategija: " << diff_irasu_dalijimo.count()
00111             << " s" << endl;
00112             diff_irasu_dalijimo1 = diff_irasu_dalijimo.count();
00113
00114             auto start_irasymas_i_vargsiuku_faila = high_resolution_clock::now();
00115             stringstream vargsiukai_buf;
00116             if (spr == "vidurki") {
00117                 vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
00118                 left << "Galutinis (Vid.)" << endl;
00119                 for (int i = 0; i < 58; i++) vargsiukai_buf << "-";
00120             }
00121             else if (spr == "mediana") {
00122                 vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
00123                 left << "Galutinis (Med.)" << endl;
00124                 for (int i = 0; i < 58; i++) vargsiukai_buf << "-";
00125             }
00126             if (spr == "abu") {
00127                 vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
00128                 left << "Galutinis (Vid.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00129                 for (int i = 0; i < 76; i++) vargsiukai_buf << "-";
00130             }
00131
00132             vargsiukai_buf << endl;
00133             for (const auto& s : vargsiukai) {
00134                 if (spr == "vidurki") {
00135                     vargsiukai_buf << setw(20) << left << s.vardas()
00136                     << setw(20) << left << s.pavarde()
00137                     << setw(18) << left << fixed << setprecision(2) << s.galutinis() << endl;
00138                 }
00139                 else if (spr == "mediana") {
00140                     vargsiukai_buf << setw(20) << left << s.vardas()
00141                     << setw(20) << left << s.pavarde()
00142                     << setw(18) << left << fixed << setprecision(2) << s.mediana_galutinis() << endl;
00143                 }
00144                 else if (spr == "abu") {
00145                     vargsiukai_buf << setw(20) << left << s.vardas()
00146                     << setw(20) << left << s.pavarde()
00147                     << setw(18) << left << fixed << setprecision(2) << s.galutinis() << setw(18) << left <<
00148                     fixed << setprecision(2) << s.mediana_galutinis() << endl;
00149                 }
00150             }
00151             string failo_vardas1 = "vargsiukai_class" + to_string(k) + ".txt";
00152             ofstream out_vargsiukai(failo_vardas1);
00153             out_vargsiukai << vargsiukai_buf.str();
00154             out_vargsiukai.close();
00155             duration<double> diff_irasymas_i_vargsiuku_faila = high_resolution_clock::now() -
00156             start_irasymas_i_vargsiuku_faila;
00157             cout << k << " irasu irasymo i vargsiuku faila laikas: " <<
00158             diff_irasymas_i_vargsiuku_faila.count() << " s" << endl;
00159             diff_irasymas_i_vargsiuku_faila1 = diff_irasymas_i_vargsiuku_faila.count();
00160
00161             auto start_irasymas_i_kietiakuku_faila = high_resolution_clock::now();
00162             stringstream kietiakai_buf;
00163             if (spr == "vidurki") {
00164                 kietiakai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left
00165                 << "Galutinis (Vid.)" << endl;
00166                 for (int i = 0; i < 58; i++) kietiakai_buf << "-";
00167             }

```



```

00161         else if (spr == "mediana") {
00162             kietiakai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left
00163             << "Galutinis (Med.)" << endl;
00164             for (int i = 0; i < 58; i++) kietiakai_buf << "-";
00165         }
00166         if (spr == "abu") {
00167             kietiakai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left
00168             << "Galutinis (Vid.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00169             for (int i = 0; i < 76; i++) kietiakai_buf << "-";
00170         }
00171
00172         kietiakai_buf << endl;
00173         for (const auto& s : kietiakai) {
00174             if (spr == "vidurki") {
00175                 kietiakai_buf << setw(20) << left << s.vardas()
00176                 << setw(20) << left << s.pavarde()
00177                 << setw(18) << left << fixed << setprecision(2) << s.galutinis() << endl;
00178             }
00179             else if (spr == "mediana") {
00180                 kietiakai_buf << setw(20) << left << s.vardas()
00181                 << setw(20) << left << s.pavarde()
00182                 << setw(18) << left << fixed << setprecision(2) << s.mediana_galutinis() << endl;
00183             }
00184             else if (spr == "abu") {
00185                 kietiakai_buf << setw(20) << left << s.vardas()
00186                 << setw(20) << left << s.pavarde()
00187                 << setw(18) << left << fixed << setprecision(2) << s.galutinis() << setw(18) << left <<
00188                 fixed << setprecision(2) << s.mediana_galutinis() << endl;
00189             }
00190             string failo_vardas2 = "kietiakai_class" + to_string(k) + ".txt";
00191             ofstream out_kietiakai(failo_vardas2);
00192             out_kietiakai << kietiakai_buf.str();
00193             out_kietiakai.close();
00194             duration<double> diff_irasymas_i_kietiakui_faila = high_resolution_clock::now() -
00195             start_irasymas_i_kietiakui_faila;
00196             cout << k << " irasu irasymo i kietiakui faila laikas: " << diff_irasymas_i_kietiakui_faila.count()
00197             << " s" << endl;
00198             diff_irasymas_i_kietiakui_faila1 = diff_irasymas_i_kietiakui_faila.count();
00199
00200             cout << "Surusiuta studentu:" << endl;
00201             cout << "Vargsai (< 5.0): " << vargsiukai.size() << " studentai" << endl;
00202             cout << "Kietiakai (>= 5.0): " << kietiakai.size() << " studentai" << endl;
00203         }
00204
00205         // 2 strategija
00206         else if (rusiavimo_strategija == 2) {
00207             vargsiukai.clear();
00208             kietiakai.clear();
00209             auto start_irasu_dalijimo = high_resolution_clock::now();
00210             pagalbine_funkc_2_strategija_class(Grupe);
00211             /*sort(Grupe.begin(), Grupe.end(), palyginimas_2_strategija_class);*/
00212             while (!Grupe.empty() && Grupe.back().galutinis() < 5.0) {
00213                 vargsiukai.push_back(Grupe.back());
00214                 Grupe.pop_back();
00215             }
00216             /*Grupe.shrink_to_fit();*/
00217             ar_naudoti_shrink_to_fit_class(Grupe);
00218             /*if (pasirinkimas == 1) {
00219                 sort(Grupe.begin(), Grupe.end(), palyginimas_vardas_class);
00220                 sort(vargsiukai.begin(), vargsiukai.end(), palyginimas_vardas_class);
00221             }
00222             else if (pasirinkimas == 2) {
00223                 sort(Grupe.begin(), Grupe.end(), palyginimas_pavarde_class);
00224                 sort(vargsiukai.begin(), vargsiukai.end(), palyginimas_pavarde_class);
00225             }
00226             else {
00227                 sort(Grupe.begin(), Grupe.end(), palyginimas_galutinis_class);
00228                 sort(vargsiukai.begin(), vargsiukai.end(), palyginimas_galutinis_class);
00229             }*/
00230             pagalbine_funkcija_class(Grupe, pasirinkimas);
00231             pagalbine_funkcija_class(vargsiukai, pasirinkimas);
00232
00233             duration<double> diff_irasu_dalijimo = high_resolution_clock::now() - start_irasu_dalijimo;
00234             cout << k << " irasu dalijimo i dvi grupes laikas 2 strategija: " << diff_irasu_dalijimo.count()
00235             << " s" << endl;
00236             diff_irasu_dalijimo1 = diff_irasu_dalijimo.count();
00237
00238             auto start_irasymas_i_vargsiuku_faila = high_resolution_clock::now();
00239             stringstream vargsiukai_buf;
00240             if (spr == "vidurki") {
00241                 vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
00242                 left << "Galutinis (Vid.)" << endl;
00243                 for (int i = 0; i < 58; i++) vargsiukai_buf << "-";
00244             }

```

```

00241         else if (spr == "mediana") {
00242             vargsiukai_buf « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Med.)" « endl;
00243             for (int i = 0; i < 58; i++) vargsiukai_buf « "-";
00244         }
00245         if (spr == "abu") {
00246             vargsiukai_buf « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) «
left « "Galutinis (Vid.)" « setw(18) « left « "Galutinis (Med.)" « endl;
00247             for (int i = 0; i < 76; i++) vargsiukai_buf « "-";
00248         }
00249
00250         vargsiukai_buf « endl;
00251         for (const auto& s : vargsiukai) {
00252             if (spr == "vidurki") {
00253                 vargsiukai_buf « setw(20) « left « s.vardas()
« setw(20) « left « s.pavarde()
00254                 « setw(18) « left « fixed « setprecision(2) « s.galutinis() « endl;
00255             }
00256             else if (spr == "mediana") {
00257                 vargsiukai_buf « setw(20) « left « s.vardas()
« setw(20) « left « s.pavarde()
00258                 « setw(18) « left « fixed « setprecision(2) « s.mediana_galutinis() « endl;
00259             }
00260             else if (spr == "abu") {
00261                 vargsiukai_buf « setw(20) « left « s.vardas()
« setw(20) « left « s.pavarde()
00262                 « setw(18) « left « fixed « setprecision(2) « s.galutinis() « setw(18) « left «
fixed « setprecision(2) « s.mediana_galutinis() « endl;
00263             }
00264             string failo_vardas1 = "vargsiukai_class" + to_string(k) + ".txt";
00265             ofstream out_vargsiukai(failo_vardas1);
00266             out_vargsiukai « vargsiukai_buf.str();
00267             out_vargsiukai.close();
00268             duration<double> diff_irasymas_i_vargsiuku_faila = high_resolution_clock::now() -
start_irasymas_i_vargsiuku_faila;
00269             cout « k « " irasu irasymo i vargsiuku faila laikas: " «
diff_irasymas_i_vargsiuku_faila.count() « " s" « endl;
00270             diff_irasymas_i_vargsiuku_faila1 = diff_irasymas_i_vargsiuku_faila.count();
00271
00272             auto start_irasymas_i_kietiakui_faila = high_resolution_clock::now();
00273             stringstream kietiakai_buf;
00274             if (spr == "vidurki") {
00275                 kietiakai_buf « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) « left
« "Galutinis (Vid.)" « endl;
00276                 for (int i = 0; i < 58; i++) kietiakai_buf « "-";
00277             }
00278             else if (spr == "mediana") {
00279                 kietiakai_buf « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) « left
« "Galutinis (Med.)" « endl;
00280                 for (int i = 0; i < 58; i++) kietiakai_buf « "-";
00281             }
00282             if (spr == "abu") {
00283                 kietiakai_buf « setw(20) « left « "Vardas" « setw(20) « left « "Pavarde" « setw(18) « left
« "Galutinis (Vid.)" « setw(18) « left « "Galutinis (Med.)" « endl;
00284                 for (int i = 0; i < 76; i++) kietiakai_buf « "-";
00285             }
00286
00287             kietiakai_buf « endl;
00288             for (const auto& s : Grupe) {
00289                 if (spr == "vidurki") {
00290                     kietiakai_buf « setw(20) « left « s.vardas()
« setw(20) « left « s.pavarde()
00291                     « setw(18) « left « fixed « setprecision(2) « s.galutinis() « endl;
00292                 }
00293                 else if (spr == "mediana") {
00294                     kietiakai_buf « setw(20) « left « s.vardas()
« setw(20) « left « s.pavarde()
00295                     « setw(18) « left « fixed « setprecision(2) « s.mediana_galutinis() « endl;
00296                 }
00297                 else if (spr == "abu") {
00298                     kietiakai_buf « setw(20) « left « s.vardas()
« setw(20) « left « s.pavarde()
00299                     « setw(18) « left « fixed « setprecision(2) « s.galutinis() « setw(18) « left «
fixed « setprecision(2) « s.mediana_galutinis() « endl;
00300                 }
00301                 string failo_vardas2 = "kietiakai_class" + to_string(k) + ".txt";
00302                 ofstream out_kietiakai(failo_vardas2);
00303                 out_kietiakai « kietiakai_buf.str();
00304                 out_kietiakai.close();
00305                 duration<double> diff_irasymas_i_kietiakui_faila = high_resolution_clock::now() -
start_irasymas_i_kietiakui_faila;
00306                 cout « k « " irasu irasymo i kietiakui faila laikas: " « diff_irasymas_i_kietiakui_faila.count()
« " s" « endl;
00307                 diff_irasymas_i_kietiakui_faila1 = diff_irasymas_i_kietiakui_faila.count();

```

```

00317
00318     cout << "Surusiuota studentu:" << endl;
00319     cout << "Vargsai (< 5.0): " << vargsiukai.size() << " studentai" << endl;
00320     cout << "Kietiakai (>= 5.0): " << Grupe.size() << " studentai" << endl;
00321 }
00322
00323 // 3 strategija
00324 else if (rusiavimo_strategija == 3) {
00325     vargsiukai.clear();
00326     kietiakai.clear();
00327     auto start_irasu_dalijimo = high_resolution_clock::now();
00328     auto vidurys = std::stable_partition(Grupe.begin(), Grupe.end(), partition_palyginimas_class);
00329     vargsiukai.resize(distance(vidurys, Grupe.end()));
00330     copy(vidurys, Grupe.end(), vargsiukai.begin());
00331     Grupe.erase(vidurys, Grupe.end());
00332     duration<double> diff_irasu_dalijimo = high_resolution_clock::now() - start_irasu_dalijimo;
00333     cout << k << " irasu dalijimo i dvi grupes laikas 3 strategija: " << diff_irasu_dalijimo.count()
    << " s" << endl;
00334     diff_irasu_dalijimo1 = diff_irasu_dalijimo.count();
00335
00336     ar naudoti_shrink_to_fit_class(Grupe);
00337
00338     auto start_irasymas_i_vargsiuku_faila = high_resolution_clock::now();
00339     stringstream vargsiukai_buf;
00340     if (spr == "vidurki") {
00341         vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Vid.)" << endl;
00342         for (int i = 0; i < 58; i++) vargsiukai_buf << "-";
00343     }
00344     else if (spr == "mediana") {
00345         vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Med.)" << endl;
00346         for (int i = 0; i < 58; i++) vargsiukai_buf << "-";
00347     }
00348     if (spr == "abu") {
00349         vargsiukai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) <<
left << "Galutinis (Vid.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00350         for (int i = 0; i < 76; i++) vargsiukai_buf << "-";
00351     }
00352
00353     vargsiukai_buf << endl;
00354     for (const auto& s : vargsiukai) {
00355         if (spr == "vidurki") {
00356             vargsiukai_buf << setw(20) << left << s.vardas()
<< setw(20) << left << s.pavarde()
00357             << setw(18) << left << fixed << setprecision(2) << s.galutinis() << endl;
00358         }
00359         else if (spr == "mediana") {
00360             vargsiukai_buf << setw(20) << left << s.vardas()
<< setw(20) << left << s.pavarde()
00361             << setw(18) << left << fixed << setprecision(2) << s.mediana_galutinis() << endl;
00362         }
00363         else if (spr == "abu") {
00364             vargsiukai_buf << setw(20) << left << s.vardas()
<< setw(20) << left << s.pavarde()
00365             << setw(18) << left << fixed << setprecision(2) << s.galutinis() << setw(18) << left <<
fixed << setprecision(2) << s.mediana_galutinis() << endl;
00366         }
00367     }
00368     string failo_vardas1 = "vargsiukai_class" + to_string(k) + ".txt";
00369     ofstream out_vargsiukai(failo_vardas1);
00370     out_vargsiukai << vargsiukai_buf.str();
00371     out_vargsiukai.close();
00372     duration<double> diff_irasymas_i_vargsiuku_faila = high_resolution_clock::now() -
start_irasymas_i_vargsiuku_faila;
00373     cout << k << " irasu irasymo i vargsiuku faila laikas: " <<
diff_irasymas_i_vargsiuku_faila.count() << " s" << endl;
00374     diff_irasymas_i_vargsiuku_faila1 = diff_irasymas_i_vargsiuku_faila.count();
00375
00376     auto start_irasymas_i_kietiakuku_faila = high_resolution_clock::now();
00377     stringstream kietiakai_buf;
00378     if (spr == "vidurki") {
00379         kietiakai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left
<< "Galutinis (Vid.)" << endl;
00380         for (int i = 0; i < 58; i++) kietiakai_buf << "-";
00381     }
00382     else if (spr == "mediana") {
00383         kietiakai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left
<< "Galutinis (Med.)" << endl;
00384         for (int i = 0; i < 58; i++) kietiakai_buf << "-";
00385     }
00386     if (spr == "abu") {
00387         kietiakai_buf << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left
<< "Galutinis (Vid.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00388         for (int i = 0; i < 76; i++) kietiakai_buf << "-";
00389     }
00390 }
00391
00392
00393

```

```

00394
00395     kietiakai_buf « endl;
00396     for (const auto& s : Grupe) {
00397         if (spr == "vidurki") {
00398             kietiakai_buf « setw(20) « left « s.vardas()
00399                 « setw(20) « left « s.pavarde()
00400                 « setw(18) « left « fixed « setprecision(2) « s.galutinis() « endl;
00401         }
00402         else if (spr == "mediana") {
00403             kietiakai_buf « setw(20) « left « s.vardas()
00404                 « setw(20) « left « s.pavarde()
00405                 « setw(18) « left « fixed « setprecision(2) « s.mediana_galutinis() « endl;
00406         }
00407         else if (spr == "abu") {
00408             kietiakai_buf « setw(20) « left « s.vardas()
00409                 « setw(20) « left « s.pavarde()
00410                 « setw(18) « left « fixed « setprecision(2) « s.galutinis() « setw(18) « left «
00411             fixed « setprecision(2) « s.mediana_galutinis() « endl;
00412         }
00413         string failo_vardas2 = "kietiakai_class" + to_string(k) + ".txt";
00414         ofstream out_kietiakai(failo_vardas2);
00415         out_kietiakai « kietiakai_buf.str();
00416         out_kietiakai.close();
00417         duration<double> diff_irasymas_i_kietiakui_faila = high_resolution_clock::now() -
00418         start_irasymas_i_kietiakui_faila;
00419         cout « k « " irasu irasymo i kietiakui faila laikas: " « diff_irasymas_i_kietiakui_faila.count()
00420         « " s" « endl;
00421         diff_irasymas_i_kietiakui_faila1 = diff_irasymas_i_kietiakui_faila.count();
00422
00423         cout « "Surusiuota studentu:" « endl;
00424         cout « "Vargsai (< 5.0): " « vargsiukai.size() « " studentai" « endl;
00425         cout « "Kietiakai (>= 5.0): " « Grupe.size() « " studentai" « endl;
00426     }
00427 }
00428 // Skaitomi generuoti failai
00429 template <typename T> void skaitymas_is_generuoto_failo_class(T& Grupe,
00430     const string& failo_vardas,
00431     double& diff_skaitymas1) {
00432
00433     auto start_skaitymas = high_resolution_clock::now();
00434     ifstream F(failo_vardas);
00435     if (!F) {
00436         cout « "Klaida: Nepavyko atidaryti failo " « failo_vardas « endl;
00437         exit(0);
00438     }
00439
00440     string header;
00441     getline(F, header);
00442
00443     while (F.peek() != EOF) {
00444         Studentas_klase temp;
00445         temp.skaityk_studenta_class(F);
00446         if (!F) {
00447             break;
00448         }
00449         Grupe.push_back(temp);
00450     }
00451     F.close();
00452     duration<double> diff_skaitymas = high_resolution_clock::now() - start_skaitymas;
00453     cout « "nFailo is " « Grupe.size() « " irasu nuskaitymo laikas: " « diff_skaitymas.count() « " s"
00454     « endl;
00455     diff_skaitymas1 = diff_skaitymas.count();
00456     cout « "Duomenys nuskaityti is failo. Rastas studentu skaicius: " « Grupe.size() « endl;
00457 }
00458
00459
00460
00461
00462
00463
00464 //Issaugota iki siol:
00465 //pragma once
00466 //include <vector>
00467 //include <string>
00468 //include "Studentas_klase.h"
00469 //
00470 //using std::vector;
00471 //using std::string;
00472 //
00473 //void skaitymas_is_generuoto_failo_class(vector<Studentas_klase>& Grupe,
00474 //    const string& failo_vardas,
00475 //    double& diff_skaitymas1);

```

```
00478 //
00480 //void studentu_rusiavimas_class(vector<Studentas_klase>& Grupe,
00481 //    int k,
00482 //    double& diff_rusiavimas1,
00483 //    double& diff_irasu_dalijimol,
00484 //    double& diff_irasymas_i_vargsiuku_failal,
00485 //    double& diff_irasymas_i_kietiaku_failal,
00486 //    int i,
00487 //    int rusiavimo_strategija, const string& spr);
00488 //
```

5.7 is_number.cpp Failo Nuoroda

```
#include "is_number.h"
```

Funkcijos

- bool `isNumber` (string s)

5.7.1 Funkcijos Dokumentacija

5.7.1.1 isNumber()

```
bool isNumber (
    string s)
```

5.8 is_number.h Failo Nuoroda

```
#include <string>
#include <cctype>
#include "Studentas.h"
```

Funkcijos

- bool `isNumber` (string s)

5.8.1 Funkcijos Dokumentacija

5.8.1.1 isNumber()

```
bool isNumber (
    string s)
```

5.9 is_number.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include <string>
00003 #include <cctype> //funkcijai isdigit(), nes kai kuriems kompiliatoriams neveikia
00004 #include "Studentas.h"
00005
00006 bool isNumber(string s);
```

5.10 mediana.cpp Failo Nuoroda

```
#include "mediana.h"
```

Funkcijos

- float [Rask_mediana](#) (vector< int > paz)

5.10.1 Funkcijos Dokumentacija

5.10.1.1 Rask_mediana()

```
float Rask_mediana (
    vector< int > paz)
```

5.11 mediana.h Failo Nuoroda

```
#include <vector>
#include "Studentas.h"
#include <algorithm>
```

Funkcijos

- float [Rask_mediana](#) (vector< int > paz)

5.11.1 Funkcijos Dokumentacija

5.11.1.1 Rask_mediana()

```
float Rask_mediana (
    vector< int > paz)
```

5.12 mediana.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include <vector>
00003 #include "Studentas.h"
00004 #include <algorithm>
00005
00006 float Rask_mediana(vector <int> paz);
```

5.13 myMultiFileApp.cpp Failo Nuoroda

```
#include "palyginimas.h"
#include "Generuoti_failai.h"
#include "is_number.h"
#include "mediana.h"
#include "Simple.h"
#include "Studentas.h"
#include <chrono>
#include "Studentas_klase.h"
#include "palyginimas_class.h"
#include "Generuoti_failai_class.h"
#include "Simple_class.h"
```

Funkcijos

- int [main](#) ()

5.13.1 Funkcijos Dokumentacija

5.13.1.1 main()

```
int main ()
```

5.14 palyginimas.cpp Failo Nuoroda

```
#include "palyginimas.h"
```

Funkcijos

- bool [palyginimas_vardas](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)
- bool [palyginimas_pavarde](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)
- bool [palyginimas_galutinis](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)
- bool [palyginimas_2_strategija](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)

5.14.1 Funkcijos Dokumentacija

5.14.1.1 palyginimas_2_strategija()

```
bool palyginimas_2_strategija (  
    const Studentas & pirm,  
    const Studentas & antr)
```

5.14.1.2 palyginimas_galutinis()

```
bool palyginimas_galutinis (  
    const Studentas & pirm,  
    const Studentas & antr)
```

5.14.1.3 palyginimas_pavarde()

```
bool palyginimas_pavarde (  
    const Studentas & pirm,  
    const Studentas & antr)
```

5.14.1.4 palyginimas_vardas()

```
bool palyginimas_vardas (  
    const Studentas & pirm,  
    const Studentas & antr)
```

5.15 palyginimas.h Failo Nuoroda

```
#include "Studentas.h"
```

Funkcijos

- bool [palyginimas_vardas](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)
- bool [palyginimas_pavarde](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)
- bool [palyginimas_galutinis](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)
- bool [palyginimas_2_strategija](#) (const [Studentas](#) &pirm, const [Studentas](#) &antr)

5.15.1 Funkcijos Dokumentacija

5.15.1.1 palyginimas_2_strategija()

```
bool palyginimas_2_strategija (  
    const Studentas & pirm,  
    const Studentas & antr)
```


5.15.1.2 palyginimas_galutinis()

```
bool palyginimas_galutinis (
    const Studentas & pirm,
    const Studentas & antr)
```

5.15.1.3 palyginimas_pavarde()

```
bool palyginimas_pavarde (
    const Studentas & pirm,
    const Studentas & antr)
```

5.15.1.4 palyginimas_vardas()

```
bool palyginimas_vardas (
    const Studentas & pirm,
    const Studentas & antr)
```

5.16 palyginimas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include "Studentas.h"
00003
00004 bool palyginimas_vardas(const Studentas& pirm, const Studentas& antr);
00005 bool palyginimas_pavarde(const Studentas& pirm, const Studentas& antr);
00006 bool palyginimas_galutinis(const Studentas& pirm, const Studentas& antr);
00007
00008 bool palyginimas_2_strategija(const Studentas& pirm, const Studentas& antr);
```

5.17 palyginimas_class.cpp Failo Nuoroda

```
#include "palyginimas_class.h"
```

Funkcijos

- bool [palyginimas_vardas_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [palyginimas_pavarde_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [palyginimas_galutinis_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [palyginimas_2_strategija_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [partition_palyginimas_class](#) (const [Studentas_klase](#) &s)

5.17.1 Funkcijos Dokumentacija

5.17.1.1 palyginimas_2_strategija_class()

```
bool palyginimas_2_strategija_class (
    const Studentas_klase & a,
    const Studentas_klase & b)
```

5.17.1.2 palyginimas_galutinis_class()

```
bool palyginimas_galutinis_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.17.1.3 palyginimas_pavarde_class()

```
bool palyginimas_pavarde_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.17.1.4 palyginimas_vardas_class()

```
bool palyginimas_vardas_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.17.1.5 partition_palyginimas_class()

```
bool partition_palyginimas_class (  
    const Studentas_klase & s)
```

5.18 palyginimas_class.h Failo Nuoroda

```
#include "Studentas_klase.h"
```

Funkcijos

- bool [palyginimas_vardas_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [palyginimas_pavarde_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [palyginimas_galutinis_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [palyginimas_2_strategija_class](#) (const [Studentas_klase](#) &a, const [Studentas_klase](#) &b)
- bool [partition_palyginimas_class](#) (const [Studentas_klase](#) &s)

5.18.1 Funkcijos Dokumentacija

5.18.1.1 palyginimas_2_strategija_class()

```
bool palyginimas_2_strategija_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.18.1.2 palyginimas_galutinis_class()

```
bool palyginimas_galutinis_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.18.1.3 palyginimas_pavarde_class()

```
bool palyginimas_pavarde_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.18.1.4 palyginimas_vardas_class()

```
bool palyginimas_vardas_class (  
    const Studentas_klase & a,  
    const Studentas_klase & b)
```

5.18.1.5 partition_palyginimas_class()

```
bool partition_palyginimas_class (  
    const Studentas_klase & s)
```

5.19 palyginimas_class.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once  
00002 #include "Studentas_klase.h"  
00003  
00004 bool palyginimas_vardas_class(const Studentas_klase& a, const Studentas_klase& b);  
00005 bool palyginimas_pavarde_class(const Studentas_klase& a, const Studentas_klase& b);  
00006 bool palyginimas_galutinis_class(const Studentas_klase& a, const Studentas_klase& b);  
00007 bool palyginimas_2_strategija_class(const Studentas_klase& a, const Studentas_klase& b);  
00008 bool partition_palyginimas_class(const Studentas_klase& s);
```

5.20 Simple.cpp Failo Nuoroda

```
#include "Simple.h"
```

Funkcijos

- [Studentas Stud_iv](#) (int k)
- [Studentas Stud_iv_atsitiktinai](#) (int k)

5.20.1 Funkcijos Dokumentacija

5.20.1.1 Stud_iv()

```
Studentas Stud_iv (  
    int k)
```

5.20.1.2 Stud_iv_atsitiktinai()

```
Studentas Stud_iv_atsitiktinai (  
    int k)
```

5.21 Simple.h Failo Nuoroda

```
#include "Studentas.h"  
#include <iomanip>  
#include <iostream>  
#include <fstream>  
#include <numeric>  
#include "is_number.h"  
#include "mediana.h"  
#include <cstdlib>  
#include <random>
```

Funkcijos

- Studentas Stud_iv (int k)
- Studentas Stud_iv_atsitiktinai (int k)
- template<typename T>
void skaitymas (T &Grupe)
- template<typename T>
void rasymas (T Grupe, string spr)

5.21.1 Funkcijos Dokumentacija

5.21.1.1 rasymas()

```
template<typename T>  
void rasymas (  
    T Grupe,  
    string spr)
```

5.21.1.2 skaitymas()

```
template<typename T>  
void skaitymas (  
    T & Grupe)
```

5.21.1.3 Stud_iv()

```
Studentas Stud_iv (
    int k)
```

5.21.1.4 Stud_iv_atsitiktinai()

```
Studentas Stud_iv_atsitiktinai (
    int k)
```

5.22 Simple.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include "Studentas.h"
00003 #include <iomanip>
00004 #include <iostream>
00005 #include <fstream>
00006 #include <numeric>
00007 #include "is_number.h"
00008 #include "mediana.h"
00009 #include <cstdlib>
00010 #include <random>
00011
00012 using std::cout;
00013 using std::cin;
00014 using std::endl;
00015 using std::ofstream;
00016 using std::ifstream;
00017 using std::fixed;
00018 using std::setprecision;
00019 using std::left;
00020 using std::right;
00021 using std::setw;
00022
00023 Studentas Stud_iv(int k);
00024 Studentas Stud_iv_atsitiktinai(int k);
00025
00026 template <typename T> void skaitymas(T& Grupe) {
00027     string failo_vardas;
00028     ifstream F;
00029     while (true) {
00030         cout << "Iveskite failo varda: ";
00031         cin >> failo_vardas;
00032         F.open(failo_vardas);
00033         if (F) {
00034             break;
00035         }
00036         else {
00037             string pasirink;
00038             while (true) {
00039                 cout << "Failo nepavyko atidaryti. Ar norite bandyti dar karta (rasykite 1), ar norite
00040 uzbaigti programa (rasykite 2)? ";
00041                 cin >> pasirink;
00042                 if (pasirink == "1") {
00043                     break;
00044                 }
00045                 else if (pasirink == "2") {
00046                     exit(0);
00047                 }
00048                 else {
00049                     cout << "Neteisingas pasirinkimas. Bandykite dar karta." << endl;
00050                 }
00051             }
00052         }
00053         string pavadinimai;
00054         getline(F, pavadinimai);
00055         Studentas Pirmas;
00056         int skait = 0;
00057         while (F.peek() != EOF) {
00058             F >> Pirmas.var >> Pirmas.pav;
00059             skait++;
```

```

00060         float mediana;
00061         int sum;
00062         while (F.peek() != '\n' && F.peek() != EOF) {
00063             int pazymys;
00064             string pazymys_pr;
00065             F >> pazymys_pr;
00066             if (isNumber(pazymys_pr)) {
00067                 if (stoi(pazymys_pr) >= 1 && stoi(pazymys_pr) <= 10) {
00068                     pazymys = stoi(pazymys_pr);
00069                     Pirmas.paz.push_back(pazymys);
00070                 }
00071             } else {
00072                 cout << "Studento nr. " << skait << " pazymiuse buvo klaida (ne sveikasis skaicius
nuo 1 iki 10): " << pazymys_pr << ". Klaida pasalinta is skaiciavimu" << endl;
00073             }
00074         }
00075         else {
00076             cout << "Studento nr. " << skait << " pazymiuse buvo klaida (ne sveikasis skaicius nuo 1
iki 10): " << pazymys_pr << ". Klaida pasalinta is skaiciavimu" << endl;
00077         }
00078     }
00079     if (Pirmas.paz.size() != 0) {
00080         Pirmas.egz = Pirmas.paz.back();
00081         Pirmas.paz.pop_back();
00082     }
00083     else {
00084         Pirmas.egz = 0;
00085     }
00086     sum = accumulate(Pirmas.paz.begin(), Pirmas.paz.end(), 0);
00087     int n;
00088     n = Pirmas.paz.size();
00089     if (Pirmas.paz.size() == 0) {
00090         Pirmas.gal = Pirmas.egz * 0.6;
00091         Pirmas.med = Pirmas.egz * 0.6;
00092     }
00093     else {
00094         Pirmas.gal = double(sum) / double(n) * 0.4 + Pirmas.egz * 0.6;
00095         mediana = Rask_mediana(Pirmas.paz);
00096         Pirmas.med = mediana * 0.4 + Pirmas.egz * 0.6;
00097     }
00098     Grupe.push_back(Pirmas);
00099     Pirmas.paz.clear();
00100 }
00101 F.close();
00102 cout << "Duomenys nuskaityti is failo. Rastas studentu skaicius: " << Grupe.size() << endl;
00103 }
00104
00105 template <typename T> void rasymas(T Grupe, string spr) {
00106     ofstream R("rezultatai.txt");
00107     if (!R) {
00108         cout << "Klaida: Nepavyko sukurti arba atidaryti failo 'rezultatai.txt'. Patikrinkite
direktorią ir teises." << endl;
00109         return;
00110     }
00111     if (spr == "vidurki") {
00112         R << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left << "Galutinis
(Vid.)" << endl;
00113         for (int i = 0; i < 58; i++) {
00114             R << "-";
00115         }
00116     }
00117     else if (spr == "mediana") {
00118         R << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left << "Galutinis
(Med.)" << endl;
00119         for (int i = 0; i < 58; i++) {
00120             R << "-";
00121         }
00122     }
00123     else if (spr == "abu") {
00124         R << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left << "Galutinis
(Vid.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00125         for (int i = 0; i < 76; i++) {
00126             R << "-";
00127         }
00128     }
00129     R << endl;
00130     if (spr == "vidurki") {
00131         for (auto Past : Grupe)
00132             R << setw(20) << left << Past.var << setw(20) << left << Past.pav << setw(18) << left << fixed <<
setprecision(2) << Past.gal << endl;
00133     }
00134     else if (spr == "mediana") {
00135         for (auto Past : Grupe)
00136             R << setw(20) << left << Past.var << setw(20) << left << Past.pav << setw(18) << left << fixed <<
setprecision(2) << Past.med << endl;
00137     }
00138 }

```

```

00139     else if (spr == "abu") {
00140         for (auto Past : Grupe)
00141             R << setw(20) << left << Past.var << setw(20) << left << Past.pav << setw(18) << left << fixed <<
setprecision(2) << Past.gal << setw(18) << left << fixed << setprecision(2) << Past.med << endl;
00142     }
00143
00144     R.close();
00145 }

```

5.23 Simple_class.cpp Failo Nuoroda

```
#include "Simple_class.h"
```

Funkcijos

- [Studentas_klase Stud_iv_class](#) (int k)
- [Studentas_klase Stud_iv_atsitiktinai_class](#) (int k)

5.23.1 Funkcijos Dokumentacija

5.23.1.1 Stud_iv_atsitiktinai_class()

```
Studentas_klase Stud_iv_atsitiktinai_class (
    int k)
```

5.23.1.2 Stud_iv_class()

```
Studentas_klase Stud_iv_class (
    int k)
```

5.24 Simple_class.h Failo Nuoroda

```

#include "Studentas_klase.h"
#include <iomanip>
#include <iostream>
#include <fstream>
#include <numeric>
#include "is_number.h"
#include "mediana.h"
#include <cstdlib>
#include <random>

```

Funkcijos

- `Studentas_klase Stud_iv_class` (int k)
- `Studentas_klase Stud_iv_atsitiktinai_class` (int k)
- `template<typename T>`
void `skaitymas_class` (T &Grupe)
- `template<typename T>`
void `rasymas_class` (T Grupe)

5.24.1 Funkcijos Dokumentacija

5.24.1.1 `rasymas_class()`

```
template<typename T>
void rasymas_class (
    T Grupe)
```

5.24.1.2 `skaitymas_class()`

```
template<typename T>
void skaitymas_class (
    T & Grupe)
```

5.24.1.3 `Stud_iv_atsitiktinai_class()`

```
Studentas_klase Stud_iv_atsitiktinai_class (
    int k)
```

5.24.1.4 `Stud_iv_class()`

```
Studentas_klase Stud_iv_class (
    int k)
```

5.25 Simple_class.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include "Studentas_klase.h"
00003 #include <iomanip>
00004 #include <iostream>
00005 #include <fstream>
00006 #include <numeric>
00007 #include "is_number.h"
00008 #include "mediana.h"
00009 #include <cstdlib>
00010 #include <random>
00011
00012 using std::cout;
00013 using std::cin;
00014 using std::endl;
00015 using std::ofstream;
00016 using std::ifstream;
```



```

00017 using std::fixed;
00018 using std::setprecision;
00019 using std::left;
00020 using std::right;
00021 using std::setw;
00022
00023 Studentas_klase Stud_iv_class(int k);
00024 Studentas_klase Stud_iv_atsitiktinai_class(int k);
00025
00026 template <typename T> void skaitymas_class(T& Grupe) {
00027     string failo_vardas;
00028     ifstream F;
00029     while (true) {
00030         cout << "Iveskite failo vardą: ";
00031         cin >> failo_vardas;
00032         F.open(failo_vardas);
00033         if (F) {
00034             break;
00035         }
00036         else {
00037             string pasirink;
00038             while (true) {
00039                 cout << "Failo nepavyko atidaryti. Ar norite bandyti dar karta (rasykite 1), ar norite
uzbaigti programa (rasykite 2)? ";
00040                 cin >> pasirink;
00041                 if (pasirink == "1") {
00042                     break;
00043                 }
00044                 else if (pasirink == "2") {
00045                     exit(0);
00046                 }
00047                 else {
00048                     cout << "Neteisingas pasirinkimas. Bandykite dar karta." << endl;
00049                 }
00050             }
00051         }
00052     }
00053     string pavadinimai;
00054     getline(F, pavadinimai);
00055     while (F.peek() != EOF) {
00056         Studentas_klase Pirmas;
00057         F >> Pirmas;
00058         Grupe.push_back(Pirmas);
00059     }
00060     F.close();
00061     cout << "Duomenys nuskaityti iš failo. Rastas studentų skaičius: " << Grupe.size() << endl;
00062 }
00063
00064 template <typename T> void rasyklas_class(T Grupe) {
00065     ofstream R("rezultatai.txt");
00066     if (!R) {
00067         cout << "Klaida: Nepavyko sukurti arba atidaryti failo 'rezultatai.txt'. Patikrinkite
direktorią ir teises." << endl;
00068         return;
00069     }
00070     R << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(18) << left << "Galutinis
(Vid.)" << setw(18) << left << "Galutinis (Med.)" << endl;
00071     for (int i = 0; i < 76; i++) {
00072         R << "-";
00073     }
00074
00075     R << endl;
00076     for (auto studentas : Grupe) {
00077         R << studentas;
00078     }
00079
00080     R.close();
00081 }

```

5.26 Studentas.h Failo Nuoroda

```

#include <string>
#include <vector>
#include <list>

```

Klasės

- struct [Studentas](#)

5.27 Studentas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include <string>
00003 #include <vector>
00004 #include <list>
00005
00006 using std::string;
00007 using std::vector;
00008 using std::list;
00009
00010 struct Studentas {
00011     string var;
00012     string pav;
00013     vector<int> paz;
00014     int egz;
00015     double gal;
00016     double med;
00017 };
```

5.28 Studentas_klase.cpp Failo Nuoroda

```
#include "Studentas_klase.h"
#include <sstream>
#include <numeric>
#include <iostream>
#include "mediana.h"
#include "is_number.h"
#include <random>
#include <iomanip>
```

Funkcijos

- std::istream & [operator>>](#) (std::istream &in, [Studentas_klase](#) &s)
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas_klase](#) &s)

5.28.1 Funkcijos Dokumentacija

5.28.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas\_klase & s)
```

5.28.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & in,
    Studentas\_klase & s)
```

5.29 Studentas_klase.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <istream>
#include <ostream>
```

Klasės

- class [Zmogus](#)
- class [Studentas_klase](#)

5.30 Studentas_klase.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include <string>
00003 #include <vector>
00004 #include <istream>
00005 #include <ostream>
00006
00007 using std::istream;
00008 using std::ostream;
00009 using std::vector;
00010 using std::string;
00011
00012 class Zmogus {
00013 protected:
00014     string var_;
00015     string pav_;
00016 public:
00017     Zmogus();
00018     /*Zmogus(istream& is);*/
00019
00020     virtual inline string vardas() const = 0;
00021     virtual inline string pavarde() const = 0;
00022
00023     virtual istream& skaityk_studenta_class(istream& is) = 0;
00024
00025     virtual ~Zmogus();
00026
00027     Zmogus(const Zmogus& z);
00028     Zmogus& operator=(const Zmogus& z);
00029 };
00030
00031
00032 class Studentas_klase : public Zmogus {
00033 private:
00034     vector<int> paz_;
00035     int egz_;
00036     double gal_;
00037     double med_;
00038 public:
00039     Studentas_klase();
00040     Studentas_klase(istream& is);
00041
00042     //Getteriai
00043     inline string vardas() const { return var_; }
00044     inline string pavarde() const { return pav_; }
00045     inline double galutinis() const { return gal_; }
00046     inline double mediana_galutinis() const { return med_; }
00047     inline const vector<int>& pazymiai() const { return paz_; }
00048     inline int egzaminas() const { return egz_; }
00049
00050     //Nuskaitymas studentas (setteris?)
00051     istream& skaityk_studenta_class(istream& is);
00052
00053     //Suskaiciuojami gal_ ir med_
00054     void skaiciuok_galutinius();
00055
00056     ~Studentas_klase();
```

```
00057     //Kopijavimo konstruktorius
00058     Studentas_klase(const Studentas_klase& s);
00059     //Kopijavimo priskirties operatorius
00060     Studentas_klase& operator=(const Studentas_klase& s);
00061
00062
00063     friend std::istream& operator>>(std::istream& in, Studentas_klase& s);
00064     friend std::ostream& operator<<(std::ostream& out, const Studentas_klase& s);
00065 };
00066
```

Rodyklė

- ~Studentas_klase
 - Studentas_klase, 9
- ~Zmogus
 - Zmogus, 11
- ar_naudoti_shrink_to_fit
 - Generuoti_failai.h, 14
- ar_naudoti_shrink_to_fit_class
 - Generuoti_failai_class.h, 23
- egz
 - Studentas, 7
- egzaminas
 - Studentas_klase, 9
- failu_generavimas
 - Generuoti_failai.cpp, 13
 - Generuoti_failai.h, 15
- gal
 - Studentas, 7
- galutinis
 - Studentas_klase, 9
- Generuoti_failai.cpp, 13
 - failu_generavimas, 13
 - partition_palyginimas, 13
- Generuoti_failai.h, 14
 - ar_naudoti_shrink_to_fit, 14
 - failu_generavimas, 15
 - pagalbine_funkc_2_strategija, 15
 - pagalbine_funkcija, 15
 - partition_palyginimas, 15
 - skaitymas_is_generuoto_failo, 15
 - studentu_rusiavimas, 16
- Generuoti_failai_class.cpp, 22
- Generuoti_failai_class.h, 23
 - ar_naudoti_shrink_to_fit_class, 23
 - pagalbine_funkc_2_strategija_class, 24
 - pagalbine_funkcija_class, 24
 - skaitymas_is_generuoto_failo_class, 24
 - studentu_rusiavimas_class, 24
- is_number.cpp, 31
 - isNumber, 31
- is_number.h, 31
 - isNumber, 31
- isNumber
 - is_number.cpp, 31
 - is_number.h, 31
- main
 - myMultiFileApp.cpp, 33
- med
 - Studentas, 7
- mediana.cpp, 32
 - Rask_mediana, 32
- mediana.h, 32
 - Rask_mediana, 32
- mediana_galutinis
 - Studentas_klase, 9
- myMultiFileApp.cpp, 33
 - main, 33
- operator<<
 - Studentas_klase, 10
 - Studentas_klase.cpp, 44
- operator>>
 - Studentas_klase, 10
 - Studentas_klase.cpp, 44
- operator=
 - Studentas_klase, 10
 - Zmogus, 12
- pagalbine_funkc_2_strategija
 - Generuoti_failai.h, 15
- pagalbine_funkc_2_strategija_class
 - Generuoti_failai_class.h, 24
- pagalbine_funkcija
 - Generuoti_failai.h, 15
- pagalbine_funkcija_class
 - Generuoti_failai_class.h, 24
- palyginimas.cpp, 33
 - palyginimas_2_strategija, 34
 - palyginimas_galutinis, 34
 - palyginimas_pavarde, 34
 - palyginimas_vardas, 34
- palyginimas.h, 34
 - palyginimas_2_strategija, 34
 - palyginimas_galutinis, 34
 - palyginimas_pavarde, 35
 - palyginimas_vardas, 35
- palyginimas_2_strategija
 - palyginimas.cpp, 34
 - palyginimas.h, 34
- palyginimas_2_strategija_class
 - palyginimas_class.cpp, 35
 - palyginimas_class.h, 36
- palyginimas_class.cpp, 35
 - palyginimas_2_strategija_class, 35
 - palyginimas_galutinis_class, 35
 - palyginimas_pavarde_class, 36

palyginimas_vardas_class, 36
 partition_palyginimas_class, 36
 palyginimas_class.h, 36
 palyginimas_2_strategija_class, 36
 palyginimas_galutinis_class, 36
 palyginimas_pavarde_class, 37
 palyginimas_vardas_class, 37
 partition_palyginimas_class, 37
 palyginimas_galutinis
 palyginimas.cpp, 34
 palyginimas.h, 34
 palyginimas_galutinis_class
 palyginimas_class.cpp, 35
 palyginimas_class.h, 36
 palyginimas_pavarde
 palyginimas.cpp, 34
 palyginimas.h, 35
 palyginimas_pavarde_class
 palyginimas_class.cpp, 36
 palyginimas_class.h, 37
 palyginimas_vardas
 palyginimas.cpp, 34
 palyginimas.h, 35
 palyginimas_vardas_class
 palyginimas_class.cpp, 36
 palyginimas_class.h, 37
 partition_palyginimas
 Generuoti_failai.cpp, 13
 Generuoti_failai.h, 15
 partition_palyginimas_class
 palyginimas_class.cpp, 36
 palyginimas_class.h, 37
 pav
 Studentas, 7
 pav_
 Zmogus, 12
 pavarde
 Studentas_klase, 10
 Zmogus, 12
 paz
 Studentas, 7
 pazymiai
 Studentas_klase, 10

 Rask_mediana
 mediana.cpp, 32
 mediana.h, 32
 rasyimas
 Simple.h, 38
 rasyimas_class
 Simple_class.h, 42

 Simple.cpp, 37
 Stud_iv, 38
 Stud_iv_atstitiktinai, 38
 Simple.h, 38
 rasyimas, 38
 skaitymas, 38
 Stud_iv, 38
 Stud_iv_atstitiktinai, 39
 Simple_class.cpp, 41
 Stud_iv_atstitiktinai_class, 41
 Stud_iv_class, 41
 Simple_class.h, 41
 rasyimas_class, 42
 skaitymas_class, 42
 Stud_iv_atstitiktinai_class, 42
 Stud_iv_class, 42
 skaiciuok_galutinius
 Studentas_klase, 10
 skaityk_studenta_class
 Studentas_klase, 10
 Zmogus, 12
 skaitymas
 Simple.h, 38
 skaitymas_class
 Simple_class.h, 42
 skaitymas_is_generuoto_failo
 Generuoti_failai.h, 15
 skaitymas_is_generuoto_failo_class
 Generuoti_failai_class.h, 24
 Stud_iv
 Simple.cpp, 38
 Simple.h, 38
 Stud_iv_atstitiktinai
 Simple.cpp, 38
 Simple.h, 39
 Stud_iv_atstitiktinai_class
 Simple_class.cpp, 41
 Simple_class.h, 42
 Stud_iv_class
 Simple_class.cpp, 41
 Simple_class.h, 42
 Studentas, 7
 egz, 7
 gal, 7
 med, 7
 pav, 7
 paz, 7
 var, 8
 Studentas.h, 43
 Studentas_klase, 8
 ~Studentas_klase, 9
 egzaminas, 9
 galutinis, 9
 mediana_galutinis, 9
 operator<<, 10
 operator>>, 10
 operator=, 10
 pavarde, 10
 pazymiai, 10
 skaiciuok_galutinius, 10
 skaityk_studenta_class, 10
 Studentas_klase, 9
 vardas, 10
 Studentas_klase.cpp, 44
 operator<<, 44

- operator>>, [44](#)
- Studentas_klase.h, [45](#)
- studentu_rusiavimas
 - Generuoti_failai.h, [16](#)
- studentu_rusiavimas_class
 - Generuoti_failai_class.h, [24](#)
- var
 - Studentas, [8](#)
- var_
 - Zmogus, [12](#)
- vardas
 - Studentas_klase, [10](#)
 - Zmogus, [12](#)
- Zmogus, [11](#)
 - ~Zmogus, [11](#)
 - operator=, [12](#)
 - pav_, [12](#)
 - pavarde, [12](#)
 - skaityk_studenta_class, [12](#)
 - var_, [12](#)
 - vardas, [12](#)
 - Zmogus, [11](#)